



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального
государственного бюджетного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _ ИУК «Информатика и управление»

КАФЕДРА ИУК5 «Системы обработки информации»

О Т Ч Е Т

УЧЕБНАЯ ПРАКТИКА

«Учебно-технологический практикум»

Лабораторная работа № 3

«Действия над массивами и строками»

Студент гр. ИУК5-13Б _____ (Амбарцумян Александр Арташесович)

(подпись) (Ф.И.О.)

Руководитель _____ (Кондратьева Светлана Дмитриевна)

(подпись) (Ф.И.О.)

Оценка руководителя _____ баллов _____

(дата)

Оценка защиты _____ баллов _____

(дата)

Оценка практики _____ баллов _____

(оценка по пятибалльной шкале)

Комиссия:

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

Калуга, 2026

Цель:

Сформировать практические навыки разработки эффективных алгоритмов обработки массивов и строк и их реализации на языке C++.

Задачи:

1. Овладеть навыками построения эффективных алгоритмов обработки массивов и строк.
2. Овладеть навыками выбора оптимальных алгоритмов программирования.
3. Овладеть навыками описания основных этапов построения алгоритмов.
4. Изобразить разработанные алгоритмы в виде блок-схем в соответствии с ГОСТ 19.701-90.
5. Реализовать алгоритмы на языке C++ и оформить отчет.

Вариант 1**№ 1**

Дана матрица $A(n \times m)$. Найти порядковые номера первого отрицательного и последнего положительного элемента (если таковые имеются). Значение элементов и их порядковые номера вывести на экран или выдать соответствующее сообщение.

Ход решения:

- 1) Ввести размеры матрицы n и m , а затем ее элементы. Матрица объявлена как двумерный массив `double a[MAX][MAX]`, где $MAX = 10$ — максимальный размер.
- 2) Порядковым номером элемента считается его номер при просмотре матрицы по строкам слева направо, начиная с 1. Для элемента $a[i][j]$ он равен $i \cdot m + j + 1$.
- 3) Переменным `firstNeg` и `lastPos` присвоить значение -1 — признак того, что нужный элемент еще не найден.
- 4) Двумя вложенными циклами `for` просмотреть все элементы. Если элемент отрицательный и `firstNeg` еще равен -1 , запомнить его номер в `firstNeg` (так запоминается только первый отрицательный элемент). Если элемент положительный, запомнить его номер в `lastPos` (номер перезаписывается, поэтому в конце останется последний положительный элемент).
- 5) Если `firstNeg` $\neq -1$, вывести элемент и его порядковый номер, иначе — сообщение об отсутствии отрицательных элементов. Аналогично для `lastPos`. Строка и столбец элемента восстанавливаются по номеру: строка = номер / m , столбец = номер mod m .

Из-за размера блок-схема разделена на две части, связанные соединителем «А».

Блок-схема алгоритма:

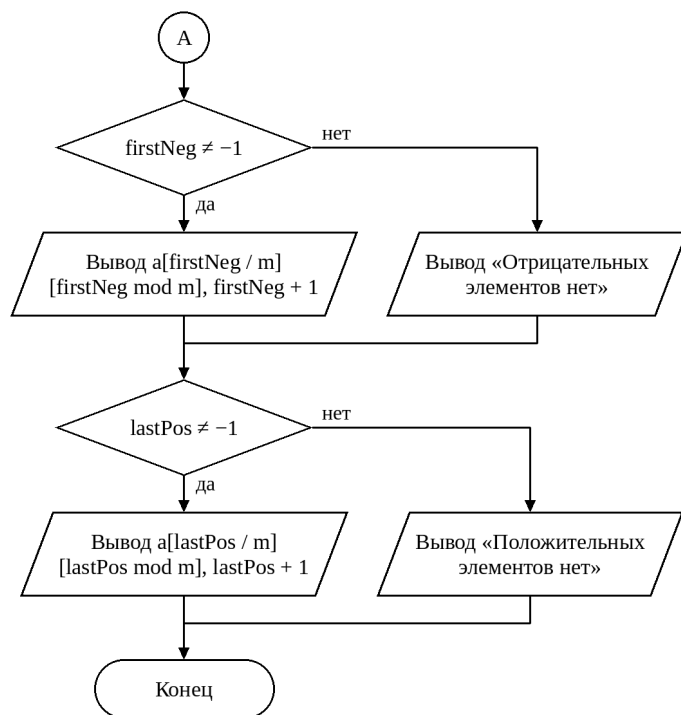
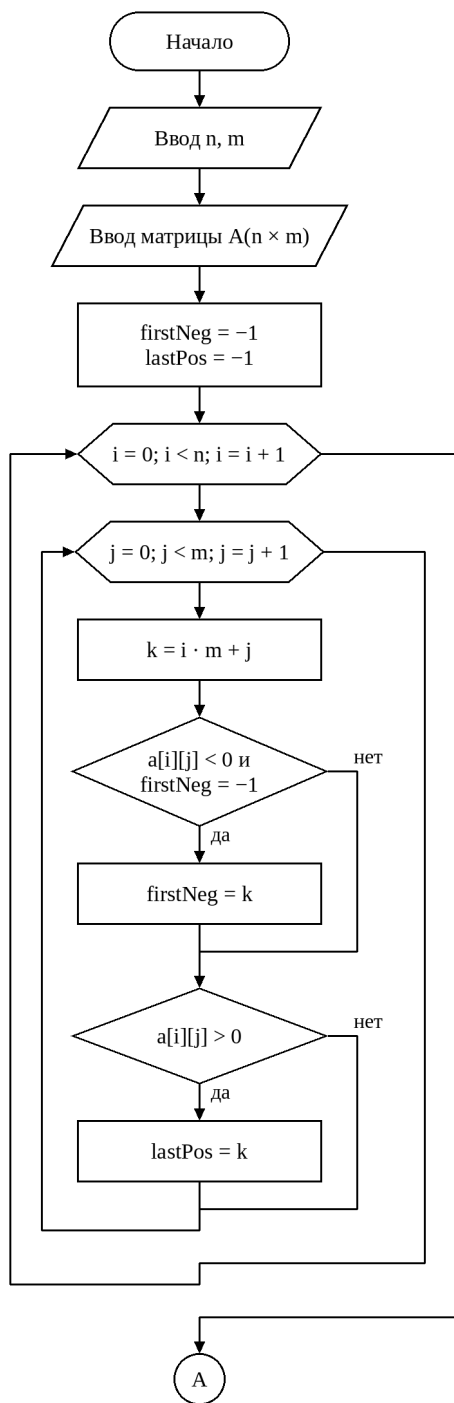


Рисунок 1 – Блок-схема алгоритма задачи № 1

Код программы:

```
1  #include <locale>
2  #include <iostream>
3
4  int main() {
5      setlocale(LC_ALL, "Russian");
6      const int MAX = 10;
7      double a[MAX][MAX];
8      int n, m;
9      std::cout << "Введите n и m (не более 10): ";
10     std::cin >> n >> m;
11     std::cout << "Введите элементы матрицы:\n";
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < m; j++) {
14             std::cin >> a[i][j];
15         }
16     }
17
18     int firstNeg = -1, lastPos = -1;
19     for (int i = 0; i < n; i++) {
20         for (int j = 0; j < m; j++) {
21             int k = i * m + j;
22             if (a[i][j] < 0 && firstNeg == -1) {
23                 firstNeg = k;
24             }
25             if (a[i][j] > 0) {
26                 lastPos = k;
27             }
28         }
29     }
30
31     if (firstNeg != -1) {
32         std::cout << "Первый отрицательный элемент: " << a[firstNeg / m][firstNeg % m]
33         << ", порядковый номер " << firstNeg + 1 << "\n";
34     } else {
35         std::cout << "Отрицательных элементов нет\n";
36     }
37     if (lastPos != -1) {
38         std::cout << "Последний положительный элемент: " << a[lastPos / m][lastPos % m]
39         << ", порядковый номер " << lastPos + 1 << "\n";
40     } else {
41         std::cout << "Положительных элементов нет\n";
42     }
43     return 0;
44 }
```

Рисунок 2 – Код программы задачи № 1

Результат работы программы:

```
Введите n и m (не более 10): 3 4
Введите элементы матрицы:
2 5 -1 0
-3 7 4 -6
1 -2 8 0
Первый отрицательный элемент: -1, порядковый номер 3
Последний положительный элемент: 8, порядковый номер 11
```

Рисунок 3 – Результат работы программы задачи № 1 (есть оба элемента)

```

Введите n и m (не более 10): 2 3
Введите элементы матрицы:
1 2 3
4 5 6
Отрицательных элементов нет
Последний положительный элемент: 6, порядковый номер 6

```

Рисунок 4 – Результат работы программы задачи № 1 (нет отрицательных)

```

Введите n и m (не более 10): 2 2
Введите элементы матрицы:
-1 -2
0 -4
Первый отрицательный элемент: -1, порядковый номер 1
Положительных элементов нет

```

Рисунок 5 – Результат работы программы задачи № 1 (нет положительных)

Проверка: в первой матрице по строкам идут элементы 2, 5, -1, 0, -3, 7, 4, -6, 1, -2, 8, 0. Первый отрицательный — -1 с номером 3, последний положительный — 8 с номером 11. Во втором и третьем примерах программа выдает соответствующие сообщения.

№ 2

Дан массив $A(n)$. Построить матрицу $A(n \times n)$ вида:

$$\begin{bmatrix} a_1 & a_2 & \dots & a_{n-1} & a_n \\ a_2 & a_3 & \dots & a_n & a_1 \\ \dots & \dots & \dots & \dots & \dots \\ a_n & a_1 & \dots & a_{n-2} & a_{n-1} \end{bmatrix}.$$

Ход решения:

- 1) Ввести n и элементы массива a .
- 2) Заметим, что каждая следующая строка матрицы — это предыдущая строка, циклически сдвинутая на один элемент влево. Поэтому элемент, стоящий в строке i и столбце j (нумерация с 0), равен $a[(i + j) \bmod n]$: остаток от деления на n «заворачивает» индекс в начало массива.
- 3) Двумя вложенными циклами заполнить матрицу $b[i][j] = a[(i + j) \bmod n]$.
- 4) Вывести матрицу построчно; для выравнивания столбцов используется манипулятор `setw(5)` из библиотеки `iomanip`.

Блок-схема алгоритма:

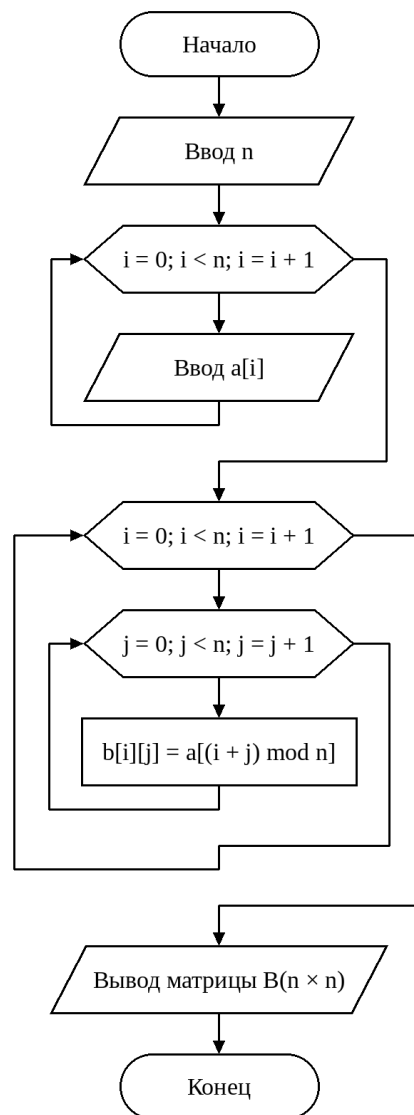


Рисунок 6 – Блок-схема алгоритма задачи № 2

Код программы:

```
1  #include <locale>
2  #include <iomanip>
3  #include <iostream>
4
5  int main() {
6      setlocale(LC_ALL, "Russian");
7      const int MAX = 10;
8      int a[MAX], b[MAX][MAX];
9      int n;
10     std::cout << "Введите n (не более 10): ";
11     std::cin >> n;
12     std::cout << "Введите элементы массива: ";
13     for (int i = 0; i < n; i++) {
14         std::cin >> a[i];
15     }
16
17     for (int i = 0; i < n; i++) {
18         for (int j = 0; j < n; j++) {
19             b[i][j] = a[(i + j) % n];
20         }
21     }
22
23     std::cout << "Полученная матрица:\n";
24     for (int i = 0; i < n; i++) {
25         for (int j = 0; j < n; j++) {
26             std::cout << std::setw(5) << b[i][j];
27         }
28         std::cout << "\n";
29     }
30     return 0;
31 }
```

Рисунок 7 – Код программы задачи № 2

Результат работы программы:

```
Введите n (не более 10): 5
Введите элементы массива: 1 2 3 4 5
Полученная матрица:
  1   2   3   4   5
  2   3   4   5   1
  3   4   5   1   2
  4   5   1   2   3
  5   1   2   3   4
```

Рисунок 8 – Результат работы программы задачи № 2 (n = 5)

```
Введите n (не более 10): 3
Введите элементы массива: 7 -2 9
Полученная матрица:
  7  -2  9
 -2  9   7
  9   7  -2
```

Рисунок 9 – Результат работы программы задачи № 2 ($n = 3$)

Проверка: при $n = 5$ вторая строка равна 2, 3, 4, 5, 1, последняя — 5, 1, 2, 3, 4, что соответствует заданному виду матрицы.

№ 3

Характеристикой столбца целочисленной матрицы $A(n \times m)$ назовем сумму модулей его отрицательных нечетных элементов. Переставляя столбцы заданной матрицы, расположить их в соответствии с ростом их характеристик.

Ход решения:

- 1) Ввести n , m и элементы целочисленной матрицы.
 - 2) Для каждого столбца j вычислить характеристику $ch[j]$: обнулить ее и просмотреть элементы столбца; если элемент отрицательный и нечетный ($a[i][j] \% 2 \neq 0$), прибавить его модуль (функция `abs()`).
 - 3) Упорядочить столбцы методом пузырька по массиву характеристик: на каждом проходе p сравниваются соседние характеристики $ch[j]$ и $ch[j + 1]$, и если $ch[j] > ch[j + 1]$, то меняются местами сами характеристики и все элементы столбцов j и $j + 1$ (функция `std::swap`). После каждого прохода самый «тяжелый» столбец оказывается в конце, поэтому внутренний цикл идет до $m - 1 - p$.
 - 4) Вывести упорядоченные характеристики и полученную матрицу.
- Блок-схема разделена на две части, связанные соединителем «А».

Блок-схема алгоритма:

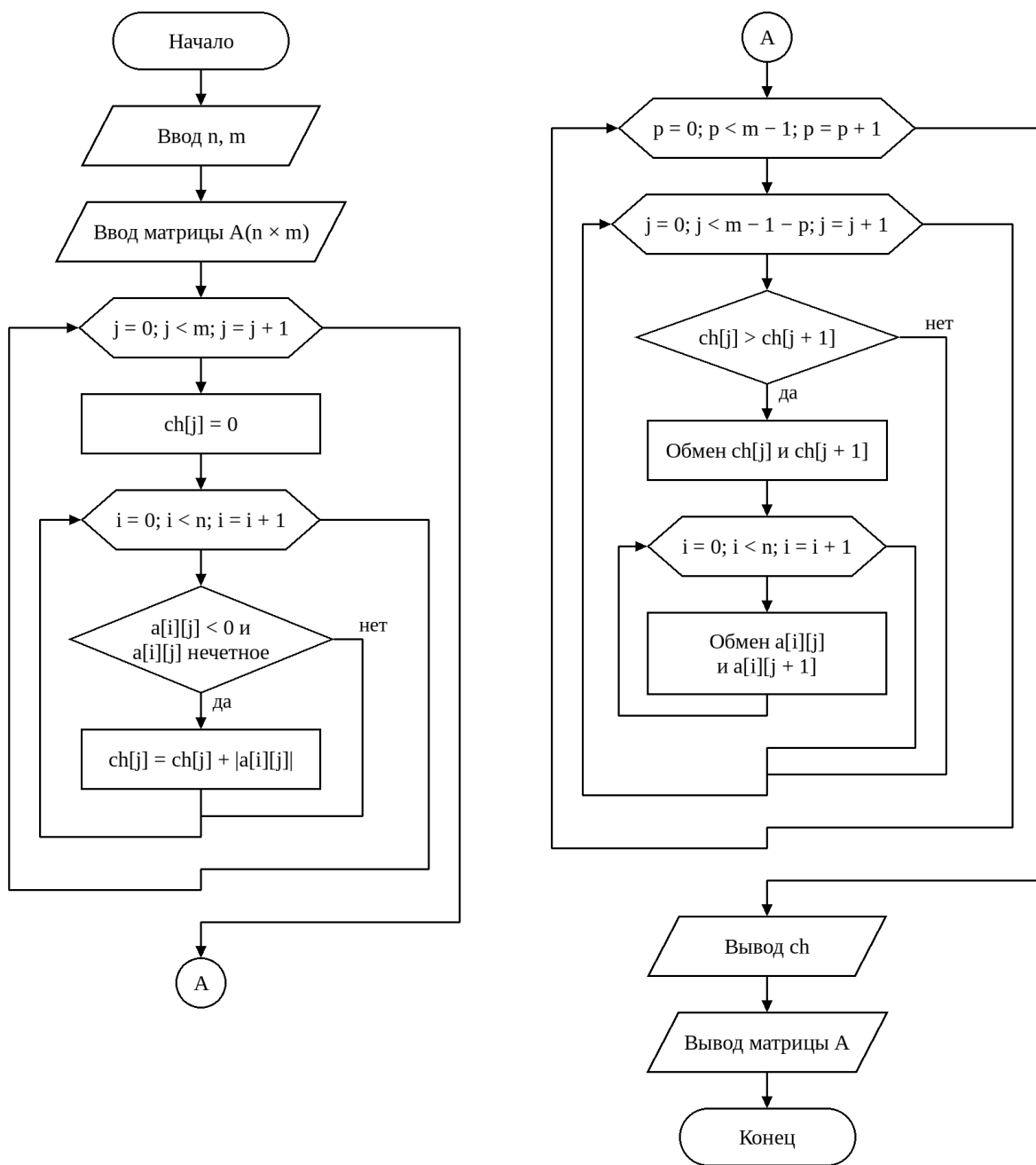


Рисунок 10 – Блок-схема алгоритма задачи № 3

Код программы:

```
1 #include <locale>
2 #include <cstdlib>
3 #include <iomanip>
4 #include <iostream>
5 #include <utility>
6
7 int main() {
8     setlocale(LC_ALL, "Russian");
9     const int MAX = 10;
10    int a[MAX][MAX], ch[MAX];
11    int n, m;
12    std::cout << "Введите n и m (не более 10): ";
13    std::cin >> n >> m;
14    std::cout << "Введите элементы матрицы:\n";
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < m; j++) {
17            std::cin >> a[i][j];
18        }
19    }
20
21    for (int j = 0; j < m; j++) {
22        ch[j] = 0;
23        for (int i = 0; i < n; i++) {
24            if (a[i][j] < 0 && a[i][j] % 2 != 0) {
25                ch[j] = ch[j] + abs(a[i][j]);
26            }
27        }
28    }
29
30    for (int p = 0; p < m - 1; p++) {
31        for (int j = 0; j < m - 1 - p; j++) {
32            if (ch[j] > ch[j + 1]) {
33                std::swap(ch[j], ch[j + 1]);
34                for (int i = 0; i < n; i++) {
35                    std::swap(a[i][j], a[i][j + 1]);
36                }
37            }
38        }
39    }
40
41    std::cout << "Характеристики столбцов:\n";
42    for (int j = 0; j < m; j++) {
43        std::cout << std::setw(5) << ch[j];
44    }
45    std::cout << "\nМатрица после перестановки столбцов:\n";
46    for (int i = 0; i < n; i++) {
47        for (int j = 0; j < m; j++) {
48            std::cout << std::setw(5) << a[i][j];
49        }
50        std::cout << "\n";
51    }
52    return 0;
53 }
```

Рисунок 11 – Код программы задачи № 3

Результат работы программы:

```
Введите n и m (не более 10): 3 4
Введите элементы матрицы:
2 -9 -3 -1
-5 4 -7 6
1 -3 8 -2
Характеристики столбцов:
      1      5     10     12
Матрица после перестановки столбцов:
-1      2     -3     -9
 6     -5     -7      4
-2      1      8     -3
```

Рисунок 12 – Результат работы программы задачи № 3 ($n = 3$, $m = 4$)

Проверка: характеристики исходных столбцов: $|-5| = 5$; $|-9| + |-3| = 12$; $|-3| + |-7| = 10$; $|-1| = 1$ (элемент -2 четный и не учитывается). По возрастанию характеристик столбцы идут в порядке 4, 1, 3, 2, что совпадает с результатом программы.

№ 4

Из текста удалить все слова заданной длины, начинающиеся с согласных букв.

Ход решения:

1) Ввести текст в символьный массив `text` функцией `cin.getline()` (она, в отличие от `cin >>`, читает строку вместе с пробелами) и длину слова `k`. Текст вводится латинскими буквами; гласными считаются буквы `a`, `e`, `i`, `o`, `u`, `y`.

2) Результат формируется в отдельном массиве `result`; `i` — индекс в исходном тексте, `г` — в результате.

3) Пока не достигнут конец строки (символ `'\0'`), проверять текущий символ. Если это не буква (пробел, знак препинания), скопировать его в `result`.

4) Если это буква — начало слова: запомнить `start = i` и сдвигать `i`, пока идут буквы. Длина слова `len = i - start`. Первая буква согласная, если ее нет в строке гласных (функция `strchr()` возвращает `NULL`).

5) Если `len = k` и слово начинается с согласной, слово не копируется, а следующие за ним пробелы пропускаются, чтобы в тексте не оставалось двойных пробелов. Иначе слово посимвольно копируется в `result`.

6) В конце дописать в `result` нулевой символ и вывести результат.

Блок-схема алгоритма:

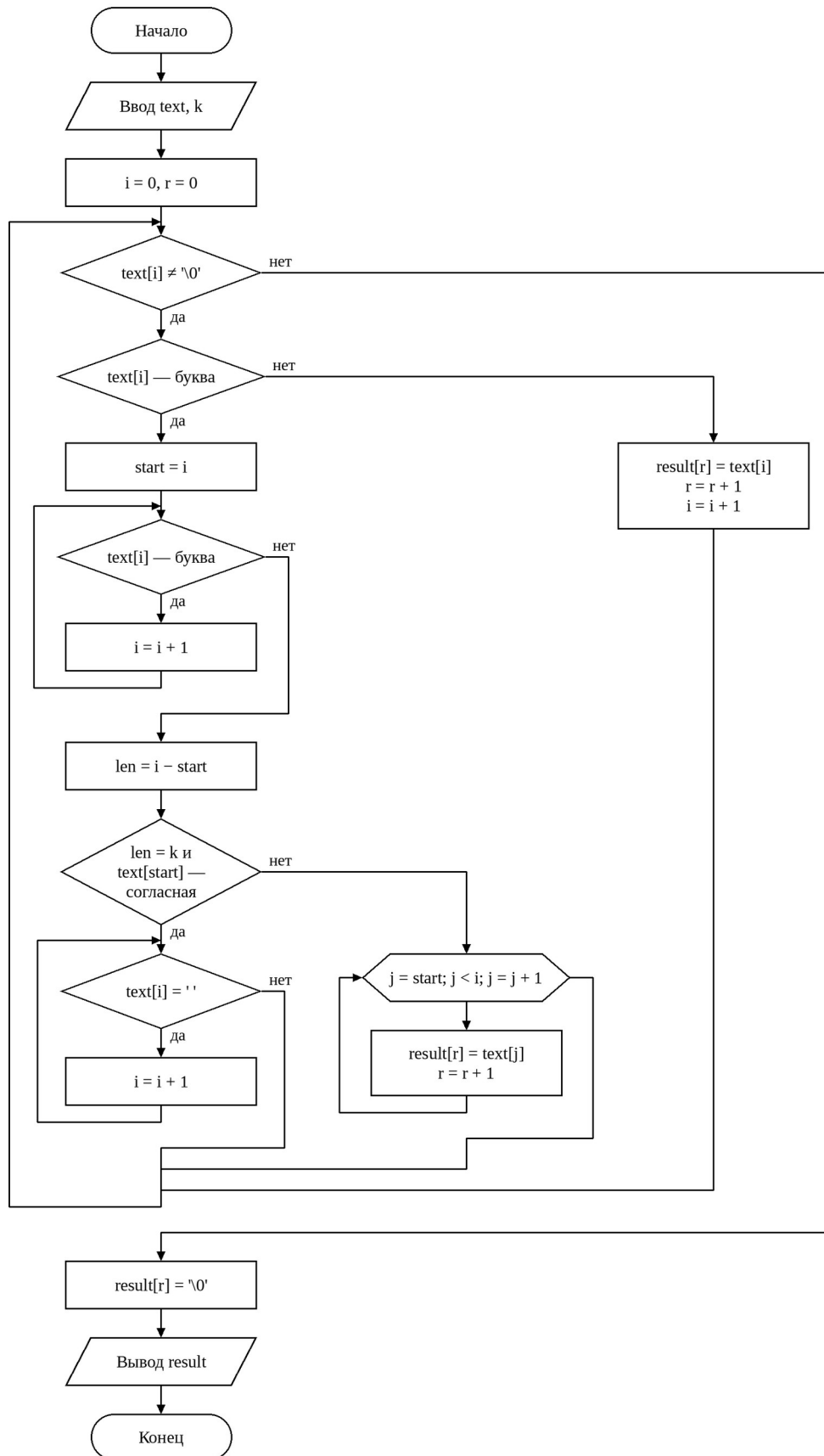


Рисунок 13 – Блок-схема алгоритма задачи № 4

Код программы:

```
1  #include <cctype>
2  #include <locale>
3  #include <cstring>
4  #include <iostream>
5
6  int main() {
7      setlocale(LC_ALL, "Russian");
8      char text[256] = "";
9      char result[256] = "";
10     int k;
11     std::cout << "Введите текст: ";
12     std::cin.getline(text, 256);
13     std::cout << "Введите длину слова: ";
14     std::cin >> k;
15
16     int i = 0, r = 0;
17     while (text[i] != '\0') {
18         if (isalpha((unsigned char)text[i])) {
19             int start = i;
20             while (isalpha((unsigned char)text[i])) {
21                 i++;
22             }
23             int len = i - start;
24             bool consonant = strchr("aeiouyAEIOUY", text[start]) == NULL;
25             if (len == k && consonant) {
26                 while (text[i] == ' ') {
27                     i++;
28                 }
29             } else {
30                 for (int j = start; j < i; j++) {
31                     result[r] = text[j];
32                     r++;
33                 }
34             }
35             result[r] = text[i];
36             r++;
37             i++;
38         }
39     }
40     result[r] = '\0';
41
42     std::cout << "Результат: " << result << "\n";
43     return 0;
44 }
45 }
```

Рисунок 14 – Код программы задачи № 4

Результат работы программы:

```
Введите текст: I like big red apples and green trees
Введите длину слова: 3
Результат: I like apples and green trees
```

Рисунок 15 – Результат работы программы задачи № 4 ($k = 3$)

```
Введите текст: The cat and the dog play in the old park
Введите длину слова: 3
Результат: and play in old park
```

Рисунок 16 – Результат работы программы задачи № 4 ($k = 3$, несколько удаляемых слов)

Проверка: в первом тексте слова длины 3 — big, red, and; удалены big и red, а слово and оставлено, так как начинается с гласной. Во втором тексте удалены The, cat, the, dog, a and и old оставлены.

№ 5

Найти, каких букв в тексте больше — гласных или согласных.

Ход решения:

- 1) Ввести текст функцией `cin.getline()`. Текст вводится латинскими буквами; гласными считаются а, е, і, о, u, у в любом регистре.
- 2) Обнулить счетчики vowels (гласные) и consonants (согласные).
- 3) В цикле `for` перебрать символы до нулевого символа. Если символ — буква (функция `isalpha()`), проверить, есть ли он в строке гласных (функция `strchr()`): если есть, увеличить vowels, иначе — consonants. Пробелы, цифры и знаки препинания не учитываются.
- 4) Вывести оба количества и сравнить их: вывести, каких букв больше, или сообщение о том, что их поровну.

Блок-схема алгоритма:

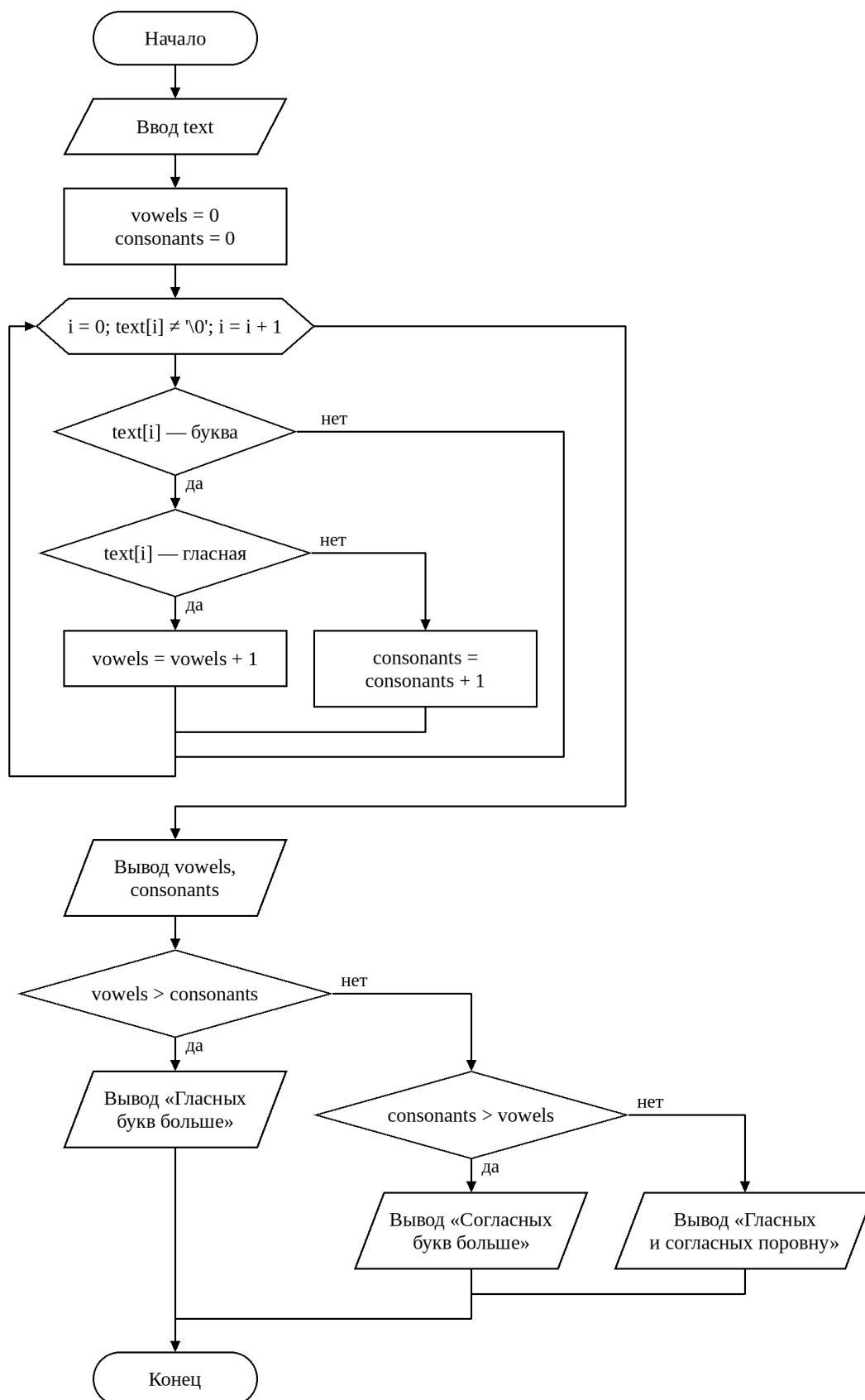


Рисунок 17 – Блок-схема алгоритма задачи № 5

Код программы:

```
1  #include <cctype>
2  #include <locale>
3  #include <cstring>
4  #include <iostream>
5
6  int main() {
7      setlocale(LC_ALL, "Russian");
8      char text[256] = "";
9      std::cout << "Введите текст: ";
10     std::cin.getline(text, 256);
11
12     int vowels = 0, consonants = 0;
13     for (int i = 0; text[i] != '\0'; i++) {
14         if (isalpha((unsigned char)text[i])) {
15             if (strchr("aeiouyAEIOUY", text[i]) != NULL) {
16                 vowels++;
17             } else {
18                 consonants++;
19             }
20         }
21     }
22
23     std::cout << "Гласных: " << vowels << ", согласных: " << consonants << "\n";
24     if (vowels > consonants) {
25         std::cout << "Гласных букв больше\n";
26     } else if (consonants > vowels) {
27         std::cout << "Согласных букв больше\n";
28     } else {
29         std::cout << "Гласных и согласных букв поровну\n";
30     }
31     return 0;
32 }
```

Рисунок 18 – Код программы задачи № 5

Результат работы программы:

```
Введите текст: Programming is fun
Гласных: 5, согласных: 11
Согласных букв больше
```

Рисунок 19 – Результат работы программы задачи № 5 (согласных больше)

```
Введите текст: I see a queue
Гласных: 8, согласных: 2
Гласных букв больше
```

Рисунок 20 – Результат работы программы задачи № 5 (гласных больше)

```
Введите текст: Data base
Гласных: 4, согласных: 4
Гласных и согласных букв поровну
```

Рисунок 21 – Результат работы программы задачи № 5 (поровну)

Проверка: в тексте «Data base» гласные a, a, a, e (4), согласные D, t, b, s (4) — поровну.

Ответы на контрольные вопросы:

1. Дайте определение понятию «массив».

Массив — это совокупность определенного количества однотипных переменных, имеющих одно имя и расположенных в памяти последовательно. Например, `int a[3]` — массив из трех переменных типа `int`.

2. Укажите, как называют переменные массива.

Переменные массива называют элементами массива.

3. Укажите, с помощью чего можно получить доступ к элементу массива.

Доступ к элементу массива выполняется по индексу — порядковому номеру элемента, который указывается в квадратных скобках после имени массива, например `a[0] = 33`. Индексация начинается с 0, поэтому последний элемент массива из `n` элементов имеет индекс `n - 1`. Для двумерного массива указываются два индекса: `a[i][j]`.

4. Приведите примеры объявления массивов.

`int a[10];` — массив из 10 целых чисел без инициализации; `int a[5] = {1, 2, 3, 4, 5};` — с инициализацией; `int a[] = {1, 2, 3};` — размер вычисляет компилятор; `int a[10] = {};` — все элементы равны 0; `int a[10] {1, 2};` — первые два элемента 1 и 2, остальные 0; `double m[3][4];` — двумерный массив (матрица) из 3 строк и 4 столбцов.

5. Раскройте значение термина «строка».

Строка в C++ — это массив символов типа `char`, последним элементом которого является нулевой символ `'\0'`. Именно нулевой символ отмечает конец строки и позволяет работать с массивом как с единым текстом.

6. Укажите различия массивов и строк.

Строка — частный случай массива: она хранит символы (тип `char`) и обязательно заканчивается символом `'\0'`. Поэтому размер массива под строку на 1 больше числа символов. Строку можно вывести целиком по имени (`cout << st`), ввести функциями `cin >>` и `cin.getline()` и обрабатывать функциями библиотеки `cstring`. Обычный массив выводится и вводится только поэлементно, в цикле.

7. Приведите примеры объявления строк.

`char st[] = "hello!";` — строка инициализируется строковой константой, `'\0'` добавляется автоматически (размер массива 7); `char st[] {'h', 'i', '\0'};` — посимвольная инициализация с явным нулевым символом; `char st[20] = "";` — пустая строка, все элементы равны `'\0'`.

8. Перечислите и опишите основные операции работы со строками.

Ввод строки: `cin >> st` (читает до первого пробела) и `cin.getline(st, размер)` (читает строку целиком вместе с пробелами); вывод: `cout << st`. Функции библиотеки `cstring`: `strlen()` — длина строки без `'\0'`; `strcpy()` — копирование одной строки в другую; `strcat()` — объединение (конкатенация) строк; `strcmp()` — сравнение строк (возвращает 0, если строки равны); `strchr()` — поиск символа в строке. Кроме того, к отдельным символам можно обращаться по индексу, как к элементам массива.

9. Укажите, что необходимо сделать, чтобы присвоить элементу массива значение `'\0'`.

Нужно обратиться к элементу по индексу и присвоить ему нулевой символ: `st[i] = '\0'`; (или `st[i] = 0;`). Чтобы сразу заполнить нулевыми символами все элементы, массив инициализируют пустой строкой: `char st[20] = "";`. Так в задаче № 4 после формирования результата выполняется `result[r] = '\0'`, чтобы обозначить конец строки.

Вывод:

В ходе выполнения лабораторной работы были сформированы практические навыки разработки алгоритмов обработки массивов и строк. Были решены задачи на поиск элементов матрицы по условию, построение матрицы из одномерного массива с использованием циклического сдвига индексов, перестановку столбцов матрицы по возрастанию их характеристик методом пузырька, удаление слов из текста и подсчет гласных и согласных букв. Для работы с текстом использованы символьные массивы, функция `cin.getline()` и функции библиотек `cstring` и `cctype`. Для всех алгоритмов построены блок-схемы в соответствии с ГОСТ 19.701-90, работа программ проверена на тестовых данных, результаты совпадают с ручным расчетом.